iTPU IT PARK UNIVERSITY

1

IT PARK UNIVERSITY

Hello! How is it going?

wunderbar

not bad

little tired but think g

norm goog

miracle did not happen

sleepy thanks to rain

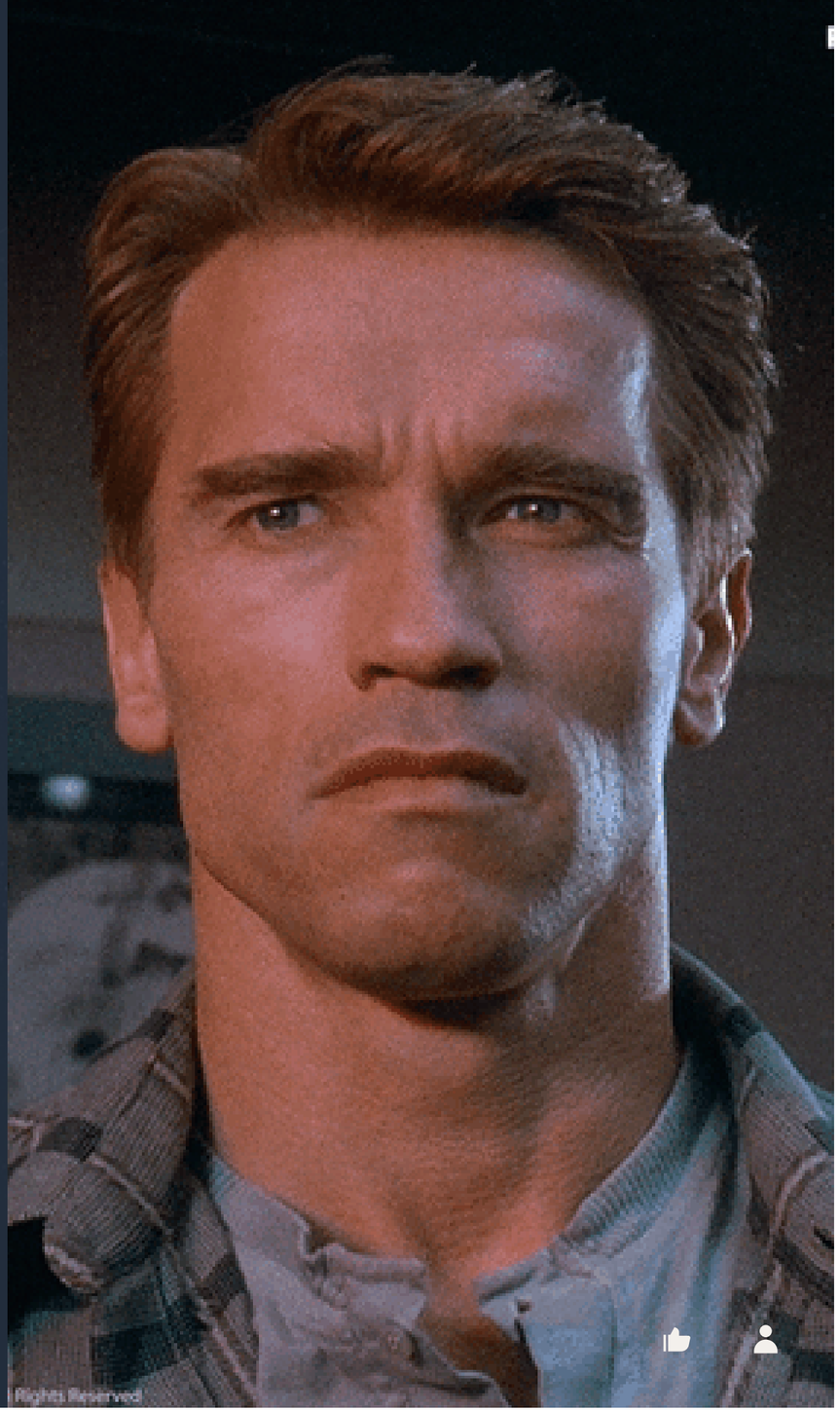
IT PARK UNIVERSITY

Agenda

- Recall last session
- Discuss current topic
- Practice
- Quiz

IT PARK UNIVERSITY

Total Recall

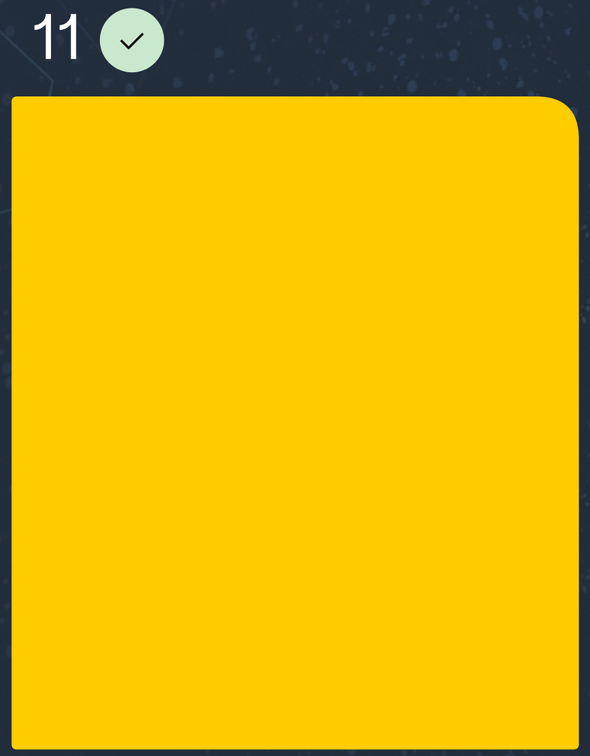


What do you remember most from few previous workshops? (short answer)

- locks
- synchronization, some fact about the Earth
- threads
- blocking synchronization
- all of suggested short answers

IT PARK UNIVERSITY

How does BlockingQueue work?



0 — Always overwrites data

11 — Prevents access if queue is full/empty

0 — Locks without exception

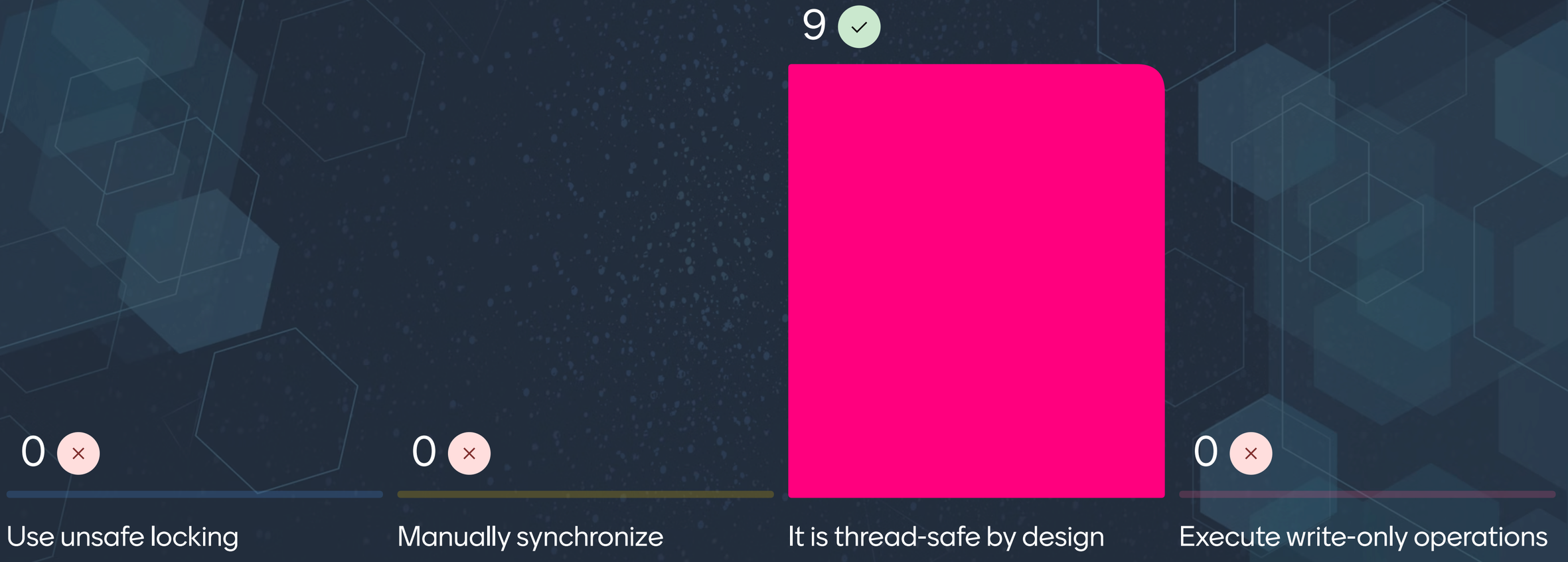
0 — Executes a priority queue

1

11 / 11

ITPU IT PARK UNIVERSITY

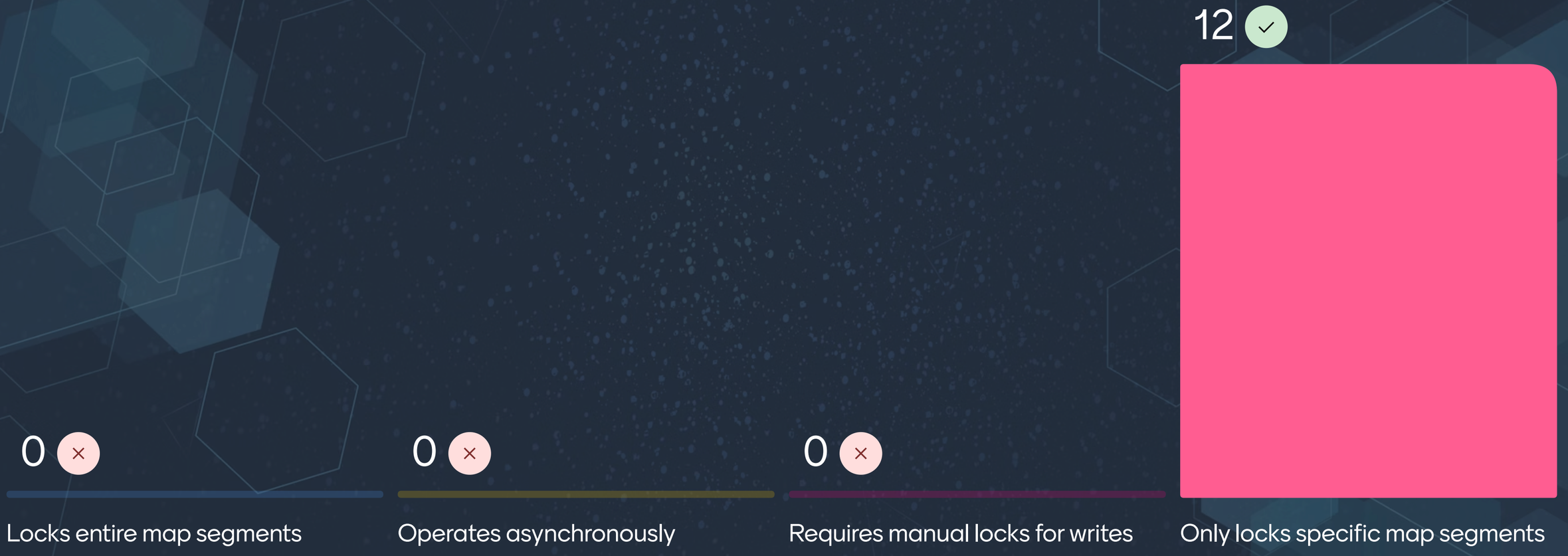
How can multiple threads access a CopyOnWriteArrayList?



1

9 / 9

What is key about ConcurrentHashMap?



ITPU IT PARK UNIVERSITY

Next topic

ITPU IT PARK UNIVERSITY

Random fact

Uzbekistan is a combination of the Turkic words “uz” (self) and “bek” (master) and the Persian suffix “-stan” (country). This essentially translates as the “Land of the Free”

iTPU IT PARK UNIVERSITY

1. Thread Control

Thread Control

- sleep() - Suspends a thread for a specified period
- join() - Makes the current thread wait for another specified thread to end
- isAlive() - Checks to see if a thread is operating
- yield() - Suspends a thread for a time quantum

sleep()

It forces a current thread to suspend execution for a specified period.

- Time is specified in milliseconds (a method with one parameter)
- it can also be specified in nanoseconds (a method with two parameters)
- it throws an exception of the InterruptedException type if a thread interrupts the one that invoked the sleep() method.

join()

- is an instance method that allows a thread to join the end of another thread. So, if thread B cannot perform its task until thread A finishes its task, B must join A, meaning that B will not be executed before the execution of A is completed.
- Sounds like: "join the current thread to the end of thread t so that t finishes before the current thread resumes":

```
main() {
  Thread t = new Thread();
  t.start();
  t.join();
}
```

main starts -> t.start() -> t.isAlive() == false -> main continue

NB! If you use the join(long timeout) option instead of the join() method without any parameters, the main thread will be paused only for the period specified.

yield()

- is a static method
- calling it for a thread currently being executed makes this thread stop for a quantum of time to let the other threads operate
- sometimes, a thread can give a quantum to itself, and suspension will not occur.

State of a Thread:

- **NEW** - a class-thread **instance before the start()**. **Considered not alive**.
- **RUNNABLE** - start() called. Thread is considered **alive**. The start() method can only be called once.
- **WAITING** -unworkable state. Waiting for another thread to perform action. State obtained after join() // wait() method **with no timeout**.
- **TIMED_WAITING -** same as WAITING but with a timeout. Gets this state after calling sleep(), join(), or wait() **with a timeout**.
- BLOCKED - unworkable state. Thread blocked waiting for access to a resource/object. Returned back to a runnable state when it gets access to it.
- TERMINATED - thread in this state is considered "dead"; its run() method is finished or stopped by another thread. An exception will be thrown if the start() method is called for a "dead" thread.

Suspention

- A thread can go into the WAITING state by calling methods such as join() and wait() with no timeout or by calling I/O methods.
- To suspend a thread, you can transfer it into the TIMED_WAITING state with the methods sleep(long millis), join(long timeout), or wait(long timeout).
- Previously, a thread could be transferred into the WAITING state by calling the suspend() method, which is now deprecated.
- A thread can be returned to the RUNNABLE state after calling the wait() method and the notify() or notifyAll() methods, and after the suspend() method by the resume() method, which is also a deprecated method.

Termination

- A thread is transferred into a "passive" TERMINATED state if the interrupt() or deprecated stop() methods are called or the run() method has finished executing and cannot be executed again. After that, the only way to use a thread is to create another thread instance.
- The interrupt() method terminates a thread successfully if it is in the RUNNABLE state. If the thread is "unworkable," for example, in the TIMED_WAITING state, the method throws an InterruptedException. To prevent this, call the isInterrupted() method in advance to check whether the thread's work has been completed.

Priorities Management

- One approach to planning the execution of several threads is time quantization, in which every thread gets enough time and is then switched to an unworkable state to give another thread a chance.
- In Java, every thread has a priority. Most JVMs use a thread planner; proactive planning based on priority allows one thread to work continuously until its run() method ends.
- Priority can vary from 1 to 10
- default Thread.NORM_PRIORITY == 5

Daemon Threads

- A daemon thread is a thread that can be executed as a background for other threads and used to maintain them—for example, as a timer sending signals to other threads at intervals. Its only use is to serve other threads.
- Daemon threads are used by applications to perform background work, but they are not an inseparable part of applications. If an operation can be executed concurrently with the main/work threads and aims to maintain them, it can be executed as a daemon thread. If only daemon threads remain active in the application, it finishes its work.

Threads and Exceptions

- threads are independent of each other while running
- the main thread will work correctly after an unexpected stop of a thread it launched for execution when an exception occurs in the thread
- the opposite is also true

Thread Interruption

A thread interruption allows a thread to stop working and do something else; the developer decides how exactly a thread reacts to an interruption, and usually, the thread is terminated.

Types:

- Catch InterruptedException - If a thread often calls methods that throw InterruptedException, the methods are terminated, and the thread catches and handles the exception. The exception handler decides whether to interrupt the thread or not.
- Check Interrupt Request - If a thread runs long without calling a method that throws InterruptedException, it should check to see if it has received an interrupt request by calling the Thread.interrupted() method from time to time. This method returns true if an interruption request is received.

java

```java
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // Re-set the flag so others know we were interrupted
    Thread.currentThread().interrupt();
    return; // Exit the task gracefully
}
```

Unlike older, unsafe methods that forcefully killed threads, interruption is a "polite nudge" that requires the thread to acknowledge the signal.

Now, explore the list of methods for managing interruptions:

- void interrupt() - Sends an interruption request to a thread. The thread interruption status is set to true. If the thread is currently paused by a call of the sleep() method, InterruptedException is generated.
- static boolean interrupted() - Checks to see if the current thread has been interrupted. The interruption status flag is then cleared to false.
- boolean isInterrupted() - Checks to see if the thread has been interrupted. Unlike the interrupted() method, this call does not change a thread's interruption status.
- static Thread currentThread() - Returns a Thread object representing the current running thread.

Every method that throws InterrupredException clears the interruption status flag.

IT PARK
UNIVERSITY

BREAK

IT PARK UNIVERSITY

2. Advanced Synchronizers

Class	Purpose	Application
Semaphore	Controls access to a common resource based on requests	If you need to limit the total number of threads that can access a resource at the same time
CyclicBarrier	Controls the wait until a certain number of threads reach a barrier point, after which the action associated with that barrier can be performed	If executing certain actions or using some results is possible only after a given number of threads have completed their tasks
CountDownLatch	Controls the run counter for threads waiting for some event	If threads need to wait for initial data or some event to start working
Exchanger	Controls data exchange between two threads at a given point	If pair threads need to share their data

The Semaphore Class

The semaphore has two types of methods:

- The acquire(), tryAcquire(), acquireUninterruptibly() methods to obtain the access permit
- The release() method to release the access permit

In real life, the semaphore is often used to message other threads about the availability of common resources. The semaphore works in scenarios where one or more tasks generate data and one or more tasks use that data

A Permit Request With Wait and Interrupt

The acquire() method requests permission to access data or execute. If there is an available permit, this method provides it to the calling thread and decrements the counter of available permits by one. If there are no available permits, the thread is locked until a permit is provided or the thread is interrupted.

A Permit Request Without Wait

The tryAcquire() method requests permission to access data or execute, with the option of not waiting for this permission if it is not in the semaphore. This method returns false if the semaphore is closed, which means the thread can do other work without waiting for permission.

A Permit Request With Wait and Without Interrupt

The acquireUninterruptibly() method requests permission to access data or execute and wait for a permit, even if the thread is interrupted. In this case, the time it will take to get permission is uncertain.

Returning an Access Permit

The thread must call the release() method to return a permit to a semaphore. This method increments the allowed permission counter by one. It is the developer's responsibility to return a permit to the semaphore correctly and in a timely manner since the acquire() and release() methods generally may not be called in the same block of code.

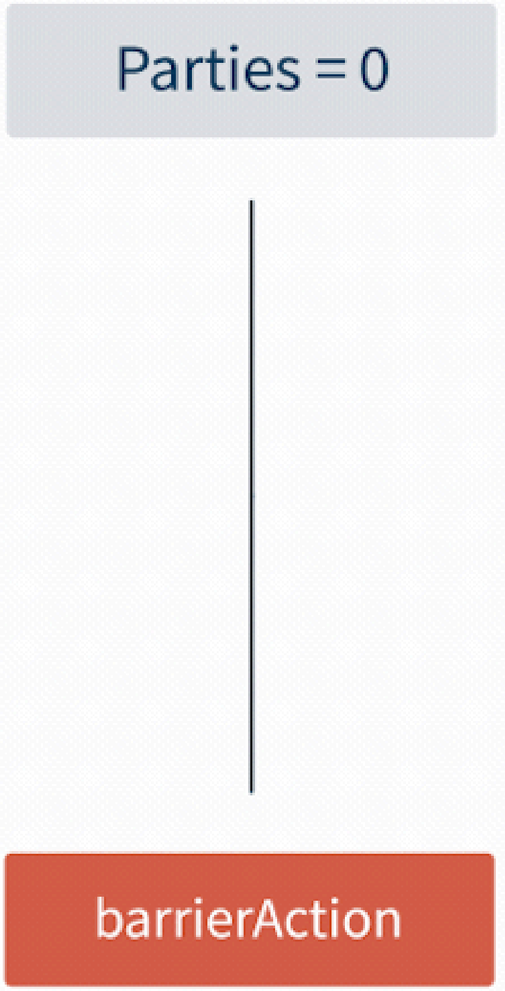
Permits = 2

The standard interaction between the `acquire()` and `release()` methods is represented by the following code. The request is wrapped in a **try** block, and the permission is released in the associated **finally** block:

```
public void run() {
    try {
        semaphore.acquire();
        // use a protected resource
    } catch (InterruptedException e) {
        // exception handling
    } finally {
        semaphore.release(); // release a semaphore
    }
}
```

The CyclicBarrier Class

All threads that reach the synchronization point will be unblocked and able to continue executing.


Parties = 0
barrierAction

await() method

Pauses the current thread until the number of threads in the barrier reaches the specified number

getParties() method

Gives the number of threads that must reach the barrier

isBroken() method

Checks the barrier state and returns true if something goes wrong

reset() method

Resets the barrier to the initial state

getNumberWaiting() method

Determines the current number of threads waiting for the barrier to open

iTPU IT PARK UNIVERSITY

The CountDownLatch Class

CountDownLatch is for waiting for events (tasks to finish), whereas CyclicBarrier is for threads to wait for each other to reach a common point.

Count = 5

Conditions:

CountDownLatch(int count)—Creating a Latch

To create a latch, the constructor with the count parameter is called and specifies the counter's initial value.

public void await()—Waiting for the Latch To Open

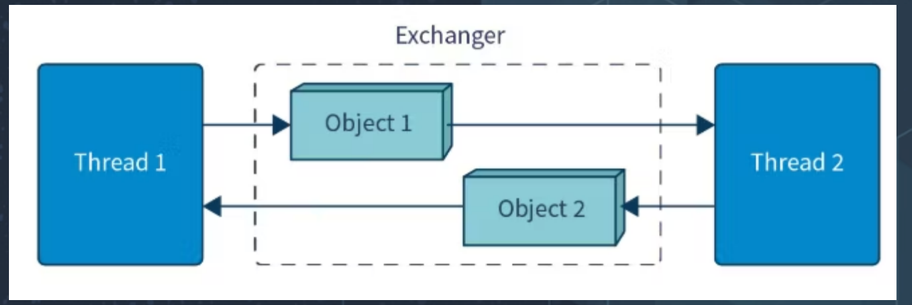
A thread that needs or wants to wait for some conditions or data to run calls the `await()` method.

public void countDown()—Preparing for the Latch to Open

Threads that deal with a latch (i.e., that prepare data or conditions) call the `countDown()` method.

The Exchanger Class

For safe data exchange between threads, including synchronized ones, you can use the Exchanger class with its only method, exchange(). The value a thread wishes to pass to another thread is specified in the method parameter, and the return value is the object the other thread is passing to the given one.



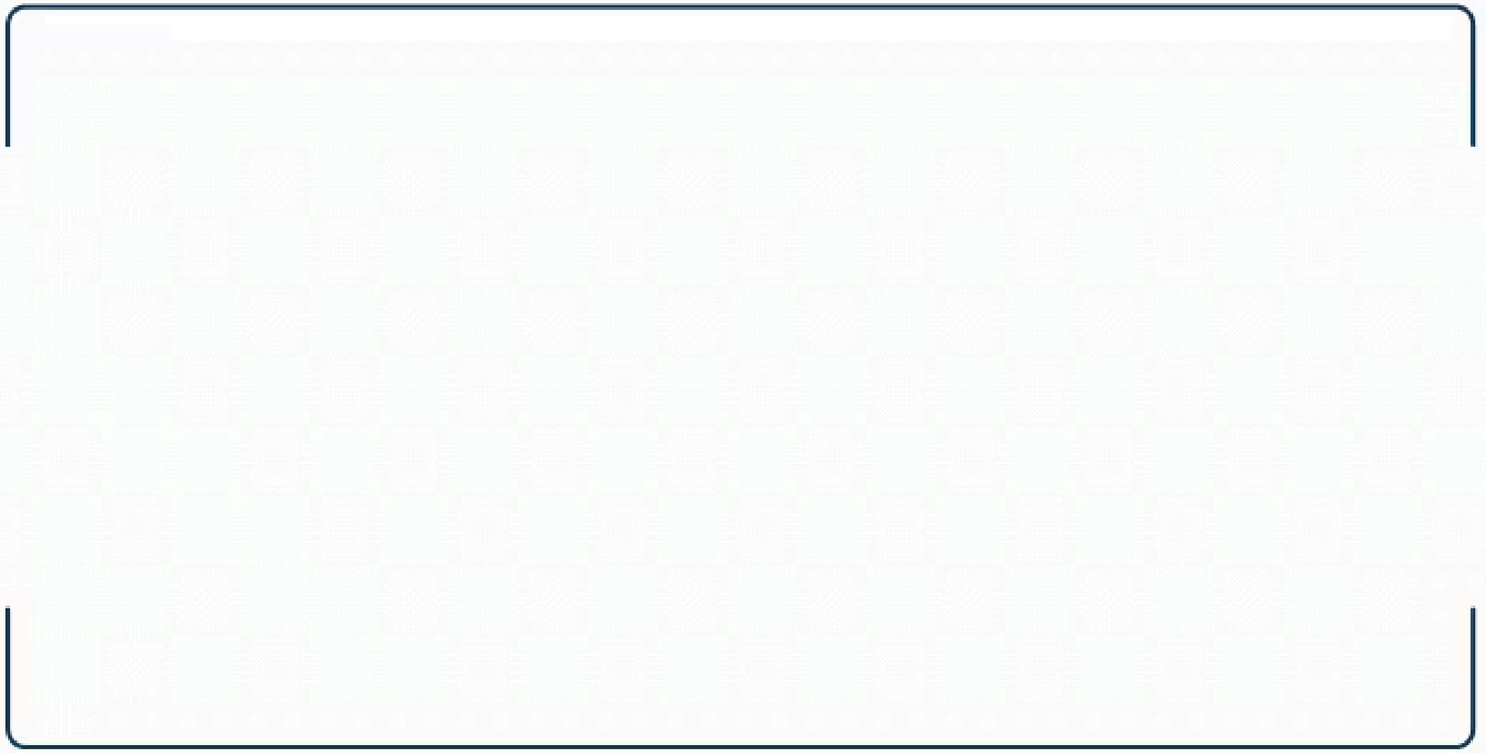
FEATURES

- Mutual Exchange
  This class ensures data transfer between threads in two directions. Exchange occurs when two threads are ready to do so. In other words, an exchange occurs when two threads on the same object of the Exchanger type have called the exchange() method.
- The exchange() Method

  To perform an exchange, a method with the following syntax is called:

  <T> exchange(<T> sendlingValue);
- Synchronization

  After calling the exchange() method, the thread goes into the "waiting" state until another thread calls the same method on the same exchange object.



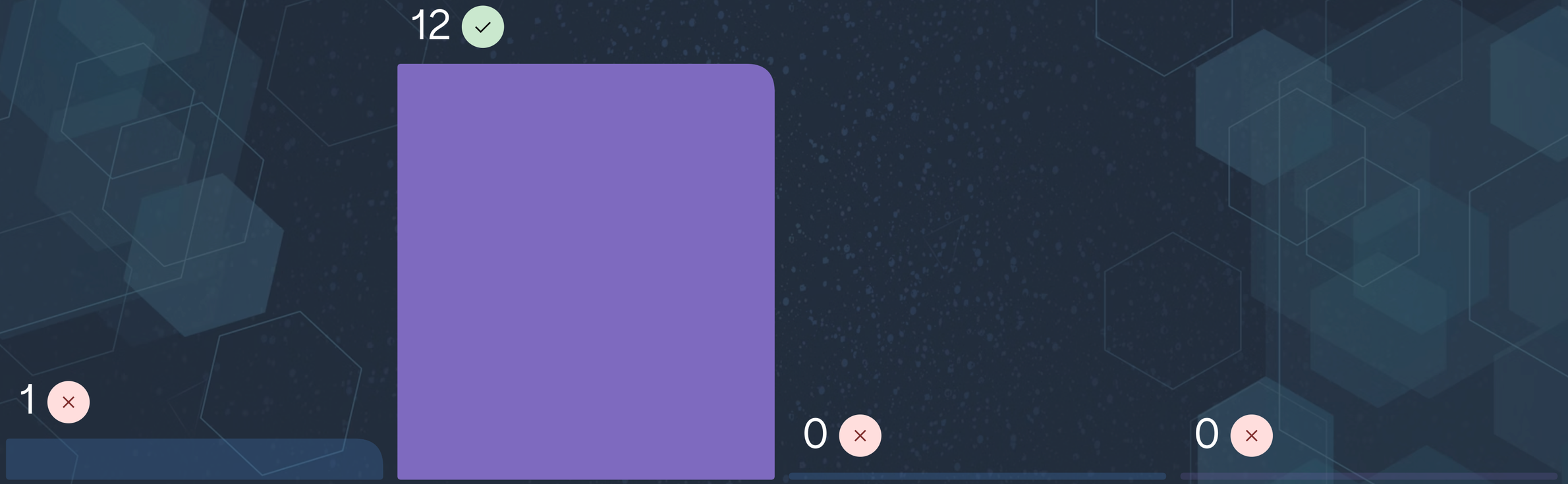
ITPU IT PARK UNIVERSITY

Quiz time

IT PARK
UNIVERSITY

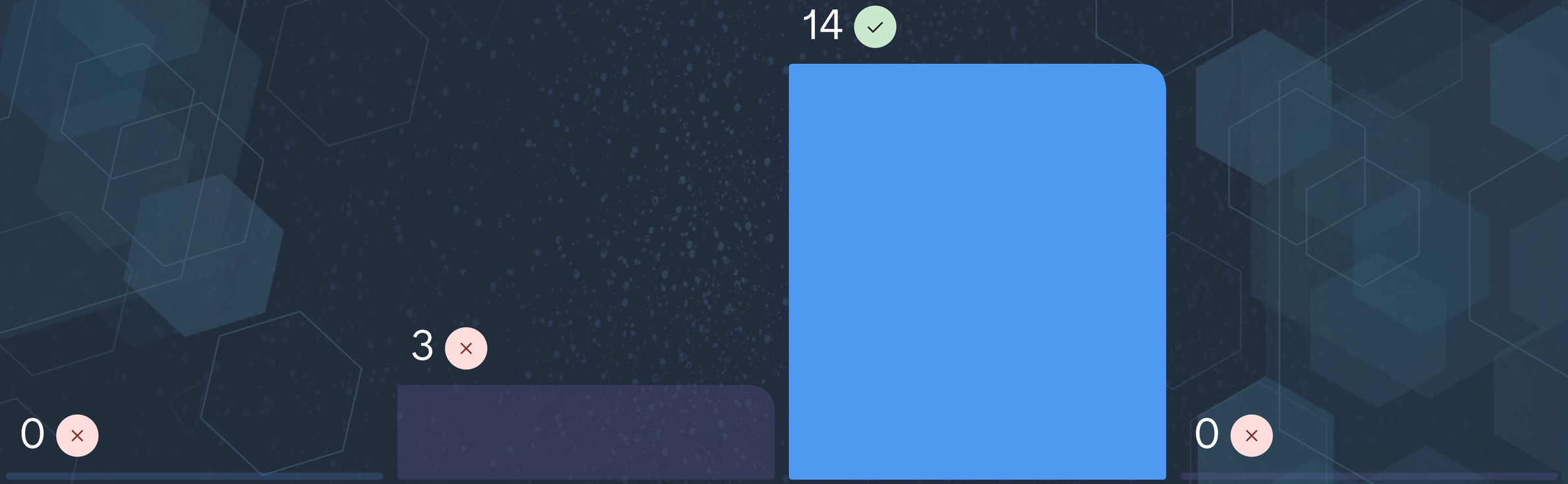
QUIZ

What does thread.join() do?



ITPU IT PARK UNIVERSITY

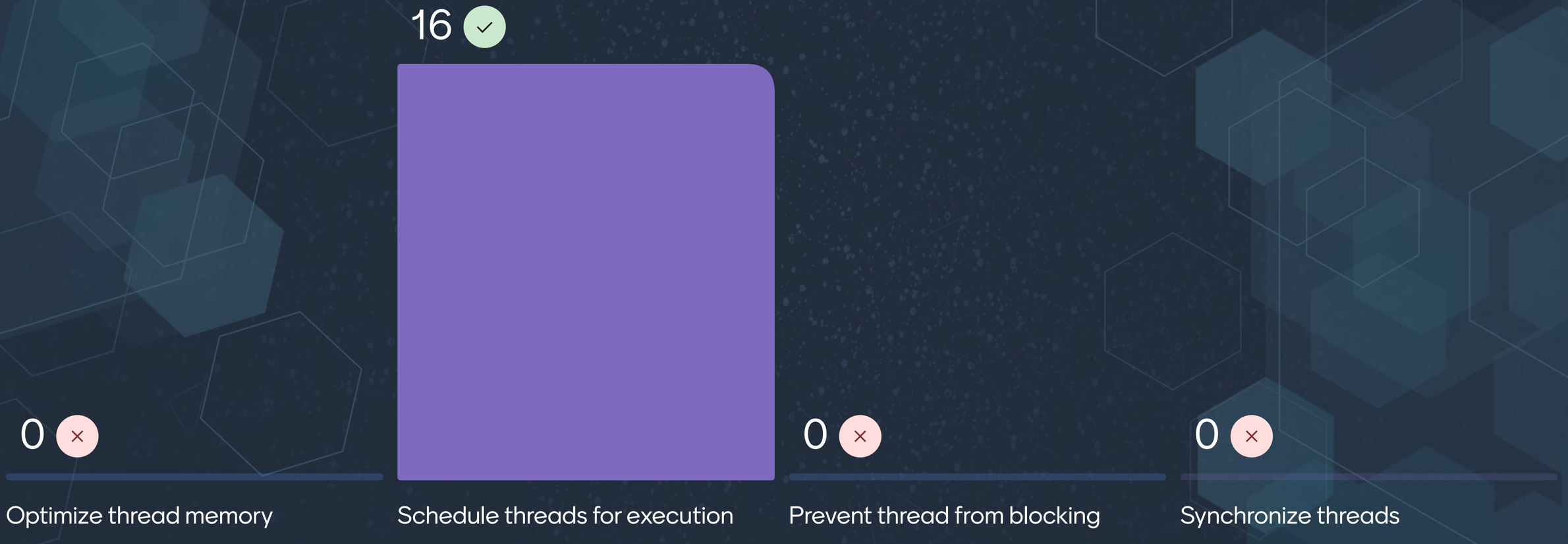
What does Thread.yield() do?



What happens to a thread after it finishes running?

iTPU IT PARK UNIVERSITY

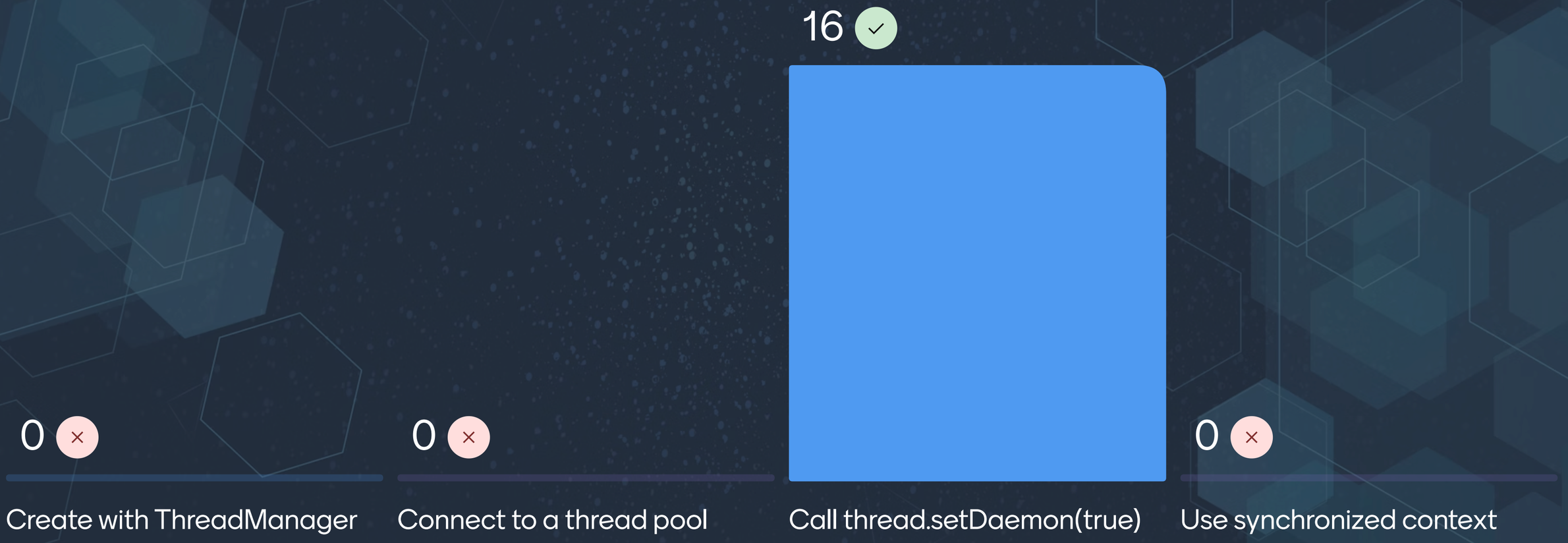
What are thread priorities used for?



4

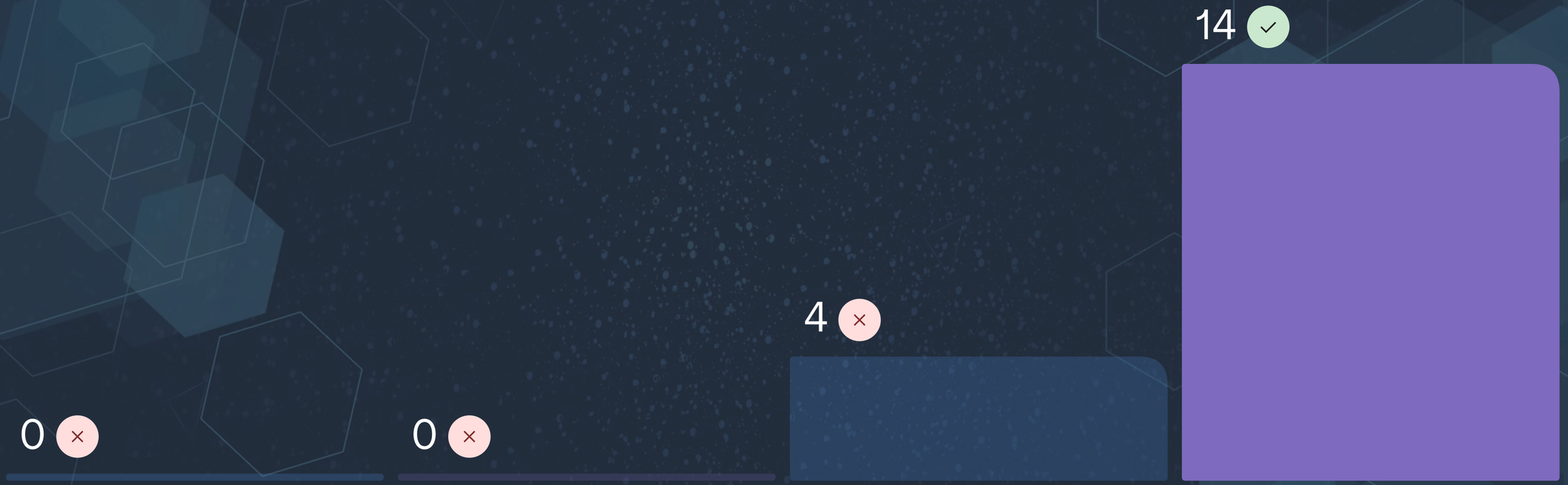
iTPU IT PARK UNIVERSITY

How do you make a thread a daemon thread?



2

What is a daemon thread?



A thread with high priority

A thread with no interruptions

A thread that runs on user demand

A low-priority background thread

iTPU IT PARK UNIVERSITY

Can a daemon thread block JVM termination?

Yes, always

Yes, with a lock

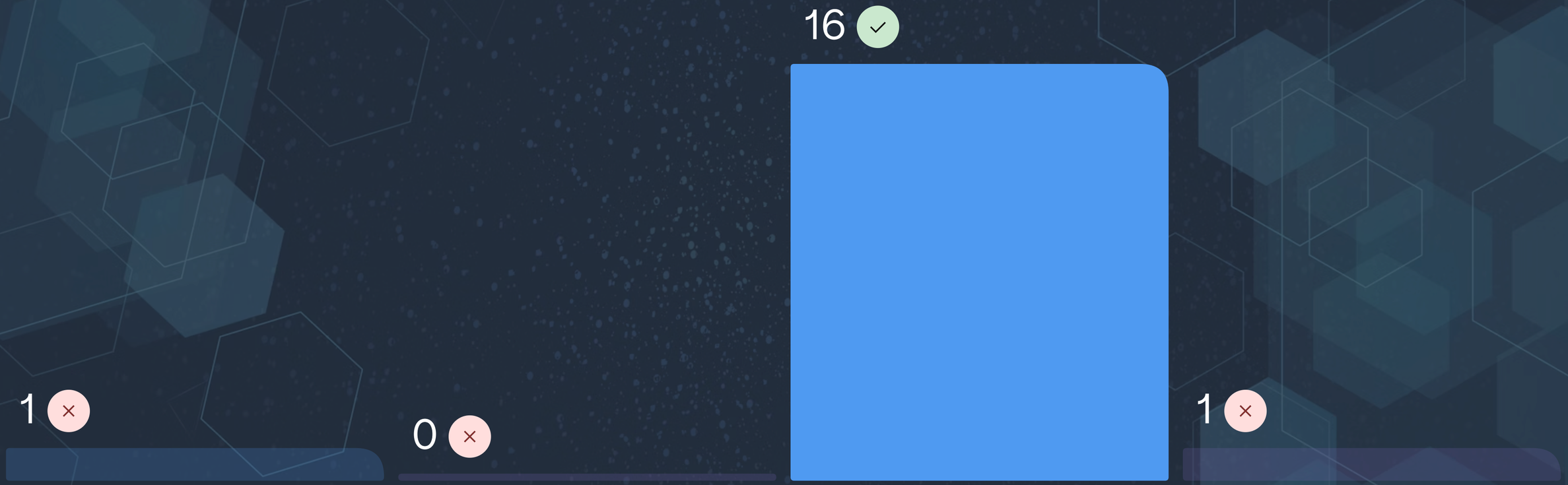
No, it pauses

No, daemon threads end with main thread

2

iTPU IT PARK UNIVERSITY

What does semaphore.acquire() do?



Adds a thread to the queue

Runs a thread fewer times

Decreases semaphore permits by 1

Synchronizes thread flow

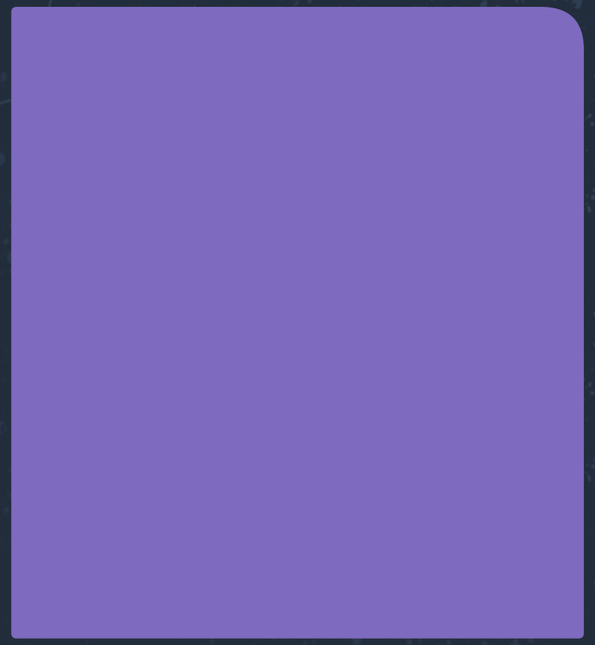
iTPU IT PARK UNIVERSITY

What does CountDownLatch do?

0 ✗

Pauses all threads indefinitely

16 ✓



Waits for a number of threads to complete

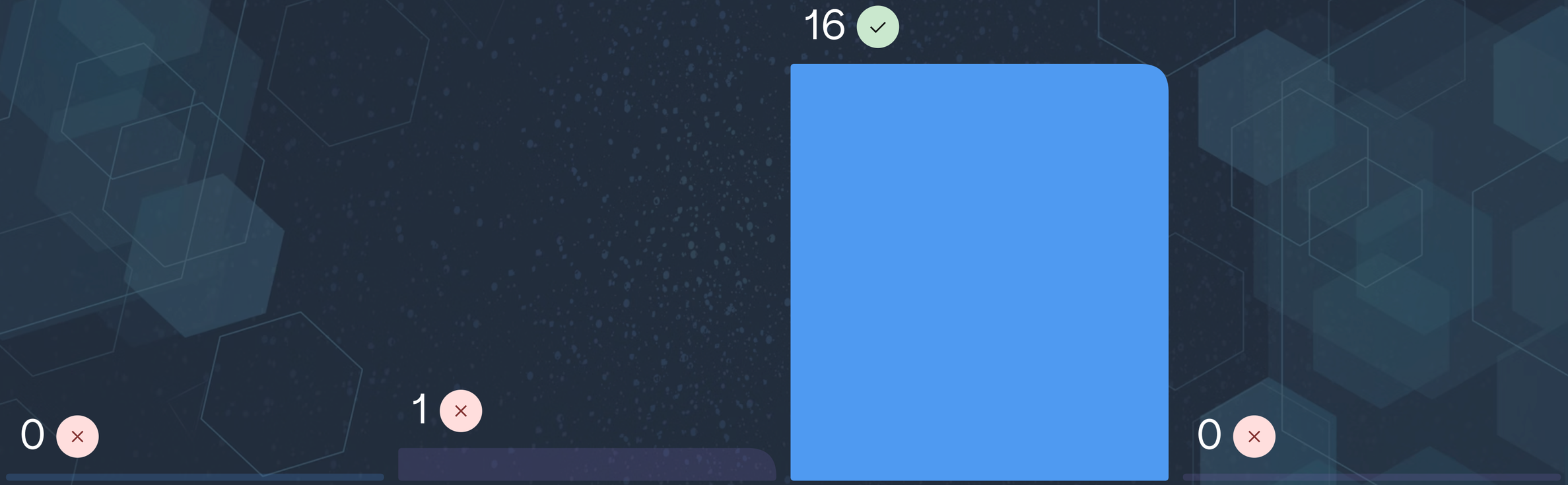
0 ✗

Returns a thread's priority

0 ✗

Terminates all waiting threads

What resets a CyclicBarrier?



IT PARK UNIVERSITY

Quiz leaderboard

IT PARK UNIVERSITY

Q&A part

1 question
0 upvotes